

From Scratch

The design and implementation of a modern SAT solver

Oliver Lie

July 30, 2026

Contents

Chapter 1	Introduction	2
1.1	What Do SAT Solvers Even Solve?	3
1.2	CNF Encoding	3

Chapter 1

Introduction

SAT solvers, in my opinion, are one of the most underrated pieces of algorithmic technology of the modern day. They are used in both software and hardware verification (circuitry testing), product configuration, package management, computational biology, cryptanalysis, particle physics, solving graph problems, and much much more.¹ In a more computer science-focused world, they have applications in NP-Completeness. Since (nearly) every problem has a reduction to a boolean SAT problem, creating one really really good SAT solver, lets us solve all those problems in a “fast” manner. So they are important everywhere despite the vast majority of us never really thinking about their existence. One note on what “fast” means: even if solving a SAT problem takes a small amount of time, the algorithmic complexity isn’t necessarily “fast” as there is no known polynomial algorithm for any NP-Complete problem (I am a firm believer that $P = NP$, and yes I know that makes me crazy).

Before we start writing out code, proofs, and the like, we need to make sure we’re all on the same page on things such as input and ideas. Reading though this introduction is important that you understand the basic notation used throughout this book. One last thing to note: we will write out pseudocode rather than the code for one specific programming language. However, although the idea behind pseudocode is that it’s programming language agnostic (it doesn’t carry artifacts of any real programming languages (usually)), this idea is poorly executed when it comes to formatting. My algorithms professor will probably scream at me for this, but I’m making the executive decision that we will follow a general C-based syntax for readability purposes. Ie, we will use `and` and `()` for block scope and grouping, `=` for assignment, `==` for equivalence, and `<=`, `>=`, etc. for their respective uses. Everything that can be inferred visually will be used. Another thing to note is that we will pretend our arrays can dynamically resize, so we won’t pre-allocate space, and we will also assume 1-based indexing for an easier map from variables to indices, as you will see later.

¹<https://www.cs.utexas.edu/~isil/cs389L/lecture4-6up.pdf>

1.1 What Do SAT Solvers Even Solve?

“What do SAT solvers even solve,” I hear you say? SAT solvers solve *boolean satisfiability* problems. Another way to say that is they solve *propositional logic* problems. There are many forms of propositional logic, and you may be familiar with it. Here are some basic examples:

- $P \rightarrow Q$, an *implication* statement
- $(A \wedge B)$, an *and* statement
- $(A \vee B)$, an *or* statement
- $\neg A$, a *negation/not* statement

Of course these are very basic examples, and we haven’t included every operator such as *xor*, *nand*, etc.. As you know, we can rewrite implications to contain only and, or, and not operators, which is what SAT solvers operate on. In other words, SAT solvers only operator on *boolean algebra* equations² However, SAT solvers don’t operate on just any boolean algebra equation. No no no, they must be in a proper form, called CNF (more on that in the next section). CNF (Conjunction Normal Form) equations are a “conjunction of disjunctions”, ie, clauses of *ors*, combined with *ands*. Some examples of CNF are as follows:

- $(A \vee B)$
- $(A \vee B) \wedge (\neg C)$
- $(A \vee B) \wedge (\neg C \vee D)$

Some examples that are not CNF:

- $\neg(A \vee B)$, where there is a NOT in front of a clause
- $\neg(A \vee B) \wedge (C)$, where is a NOT in front of a clause
- $(A \vee B \wedge C)$, where there is an AND within a clause
- $(A \wedge B) \vee (C)$, where it is in DNF form³

1.2 CNF Encoding

²A boolean algebra equation is one containing only and ($\wedge$), or ($\vee$), and not ($\neg$) operations (at a basic level).

³DNF (Disjunctive Normal Form) is a “disjunction of conjunctions”, ie clauses of *ands*, combined by *ors*. Solving these are trivially easy as we just need to find one clause where no element evaluates to false.